Methods of Artificial Intelligence

Lecture 1

Mateusz Dyndał, WFiIS KOiDC, D11/p113

Course organization

- Course theme: intro to artificial intelligence methods with the usage of Python programming language
- Lectures 30 hours
- Computer Labs 20 hours (**compulsory!**)
- Project labs 5 hours (**compulsory!**)
- Final grade –>
  Two grades – solving lab exercises: G1, project: G2
  Final Grade (FG) = 0.35*G1 + 0.65*G2. Active participation in the labs/lecture may enhance the final grade by up to 0.5

  Please make sure in advance that you have computer account with the Uni!
- Our contact details
  - ❑ Mateusz Dyndal (lectures + 2 groups for labs/project): dyndal@agh.edu.pl, D11/113
  - ❑ Rohit Salgotra (2 groups for labs/project): rohits@agh.edu.pl , D7/319

Projects

- They should be treated as an essential part of the course
- Working on the projects
  - You will work within the group of two
  - Assigning the topics should happen around one of the last computer labs
  - You may propose the topic of your project (if reasonable); otherwise there will be also a list of projects to choose
  - Passing a project: writing a summary report + short presentation during the last lecture(s) [TBD]

Literature

1. Wojciech Broniowski, Explaining neural networks in raw Python: lectures in Jupyter.
2. Francois Chollet, Deep Learning with Python, Second Edition.

Also:

3. Machine Learning for Physicists (university course), Florian Marquardt
4. Machine Learning course (IFJ-PAN), Marcin Wolter
5. CVPR 2012 Tutorial on Deep Learning Part III, Marc'Aurelio Ranzato

Software

- **Python** programming language (did you attend the python course?)
- matplotlib
- http://scikit-learn.org
  scikit-learn - Machine Learning in Python
  Simple and efficient tools for data mining and data analysis Built on NumPy, SciPy, and matplotlib
- https://keras.io/
  Keras: The Python Deep Learning library
  Emulates Deep Neural Network, uses google TensorFlow software

Course outline

- Machine learning: what does it mean?
- A little bit of mathematics and examples of simple linear classifiers.
- Principal component analysis (PCA)
- Simple non-linear methods like k-nearest neighbours
- Ensemble learning – Boosted Decision Trees (BDTs)
- Shallow neural networks
- Deep Neural Networks
- Convolutional Deep Neural Network
- Network Training - cross-validation
- Augmentation, transfer learning, optimizations, …
- Autoencoders etc.
- Generative Adversarial Networks (GANs)
- Reinforcement learning

Statistics

- Statistics –science of collecting and analyzing numerical data
- First works – أبو يوسف يعقوب بن إسحاق الصبّاح الكندي Al-Kindi (801-873) - used statistical methods to break the Caesar cipher by analyzing the frequency of letters in the text

The first page of al-Kindi's manuscript "On Deciphering Cryptographic Messages", containing the oldest known description of cryptanalysis by frequency analysis.

Caesar cipher

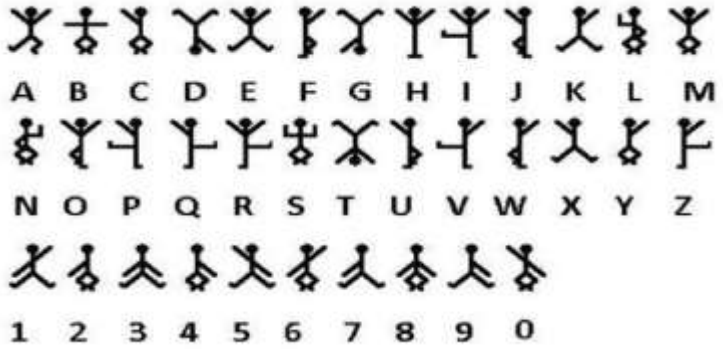
- Each letter in the text is replaced by another, but shifted by *n* letters:

Plain	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Cipher	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W

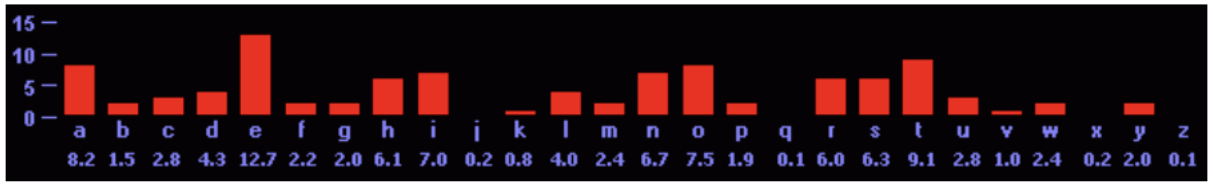
When encrypting, a person looks up each letter of the message in the "plain" line and writes down the corresponding letter in the "cipher" line.

```
Plaintext:  THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG
Ciphertext: QEB NRFZH YOLTK CLU GRJMP LSBO QEB IXWV ALD
```

- In general – replacement cipher (each letter is replaced by another, but always the same letter):



- Frequency analysis – investigation, how frequently each symbols appears

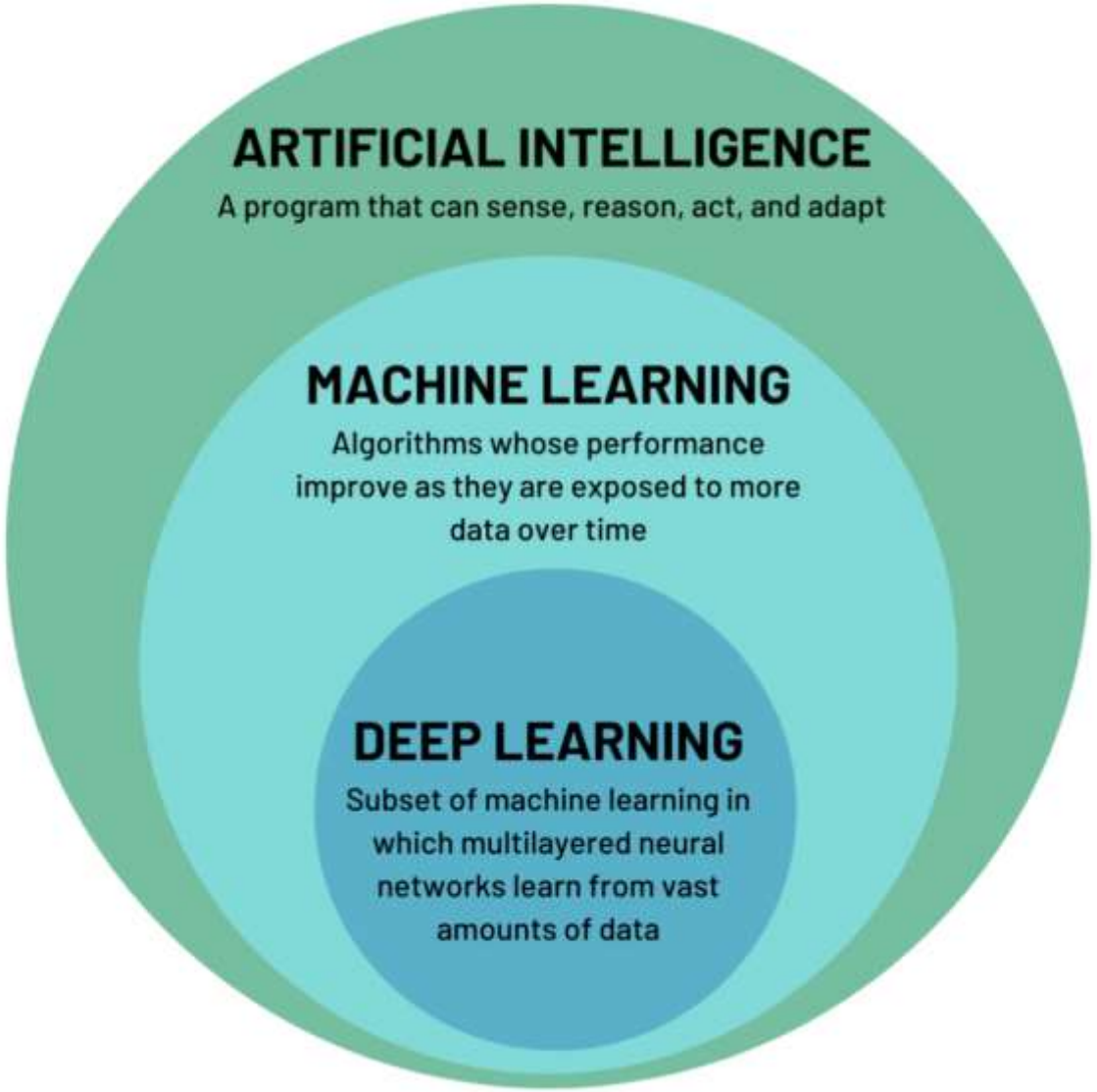


Frequency of appearance of different letters in English.

Artificial Intelligence

- **Artificial intelligence (AI)** is intelligence—perceiving, synthesizing, and inferring information—demonstrated by machines, as opposed to intelligence displayed by non-human animals or by humans. Example tasks in which this is done include speech recognition, computer vision, translation between (natural) languages, as well as other mappings of inputs.
- AI applications include **advanced web search engines (e.g., Google Search), recommendation systems (used by YouTube, Amazon, and Netflix), understanding human speech (such as Siri and Alexa), self-driving cars (e.g., Waymo), generative or creative tools (ChatGPT etc), automated decision-making, and competing at the highest level in strategic game systems (such as chess and Go).**
  (Wikipedia)

Artificial Intelligence vs. Machine Learning vs. Deep Learning

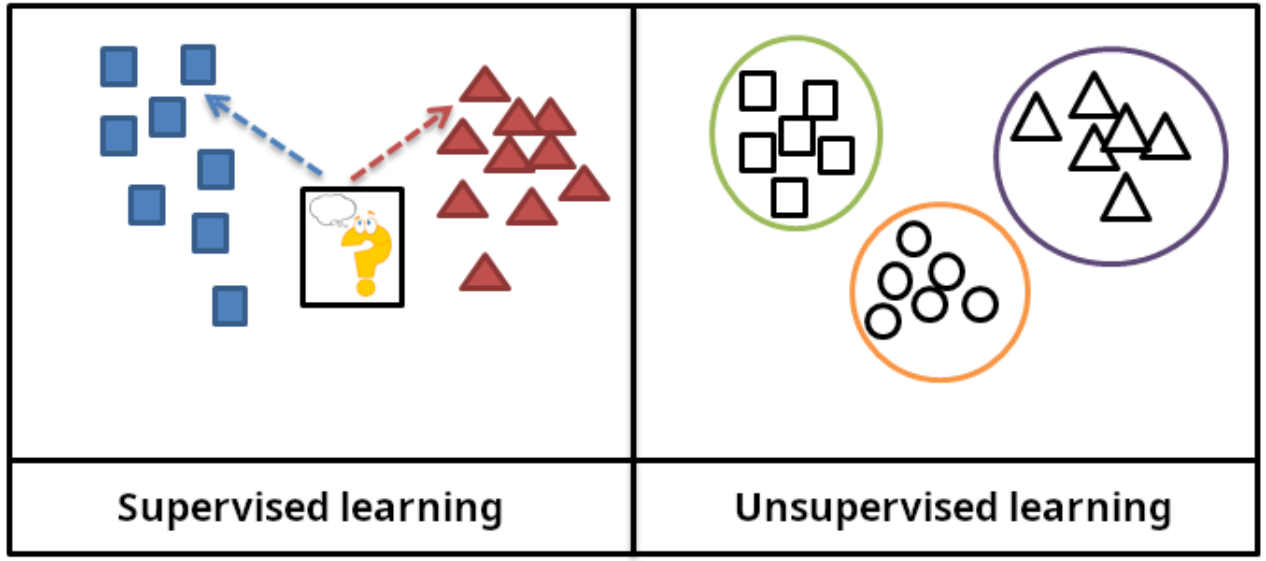


What “machine learning” really is?

- **Machine learning** is a field of computer science that gives computer systems the ability to "learn" (i.e. progressively improve performance on a specific task) with data, without being explicitly programmed.
- **Problems solved by ML:**
  - Supervised learning (classification & regression)
  - Clustering (unsupervised learning)
  - Dimensionality reduction
  - Data generation
  - Reinforcement learning
  - Many others….

(Un)supervised learning

- **“Supervised learning”**
  - Technique for creating a function from training data. The training data consist of pairs of input objects (typically vectors) and desired outputs. -> **Classification**
- **“Unsupervised learning”**
  - Technique of trying to find hidden structures in unlabeled data -> **Clustering**

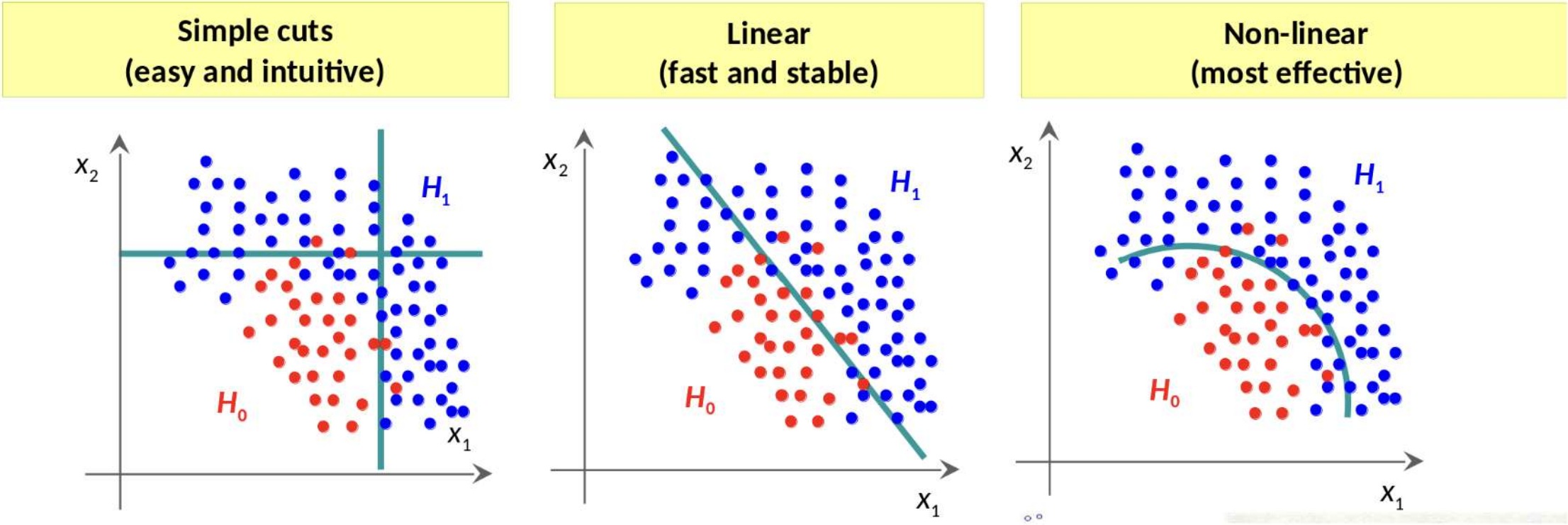


How do the (supervised) machine learning algorithms work?

- We need the **training data**, for which we know the correct answer, whether it's a signal or background. We divide the data into two samples: **training** and **test** sample.
- We have to find the best function *f(x)* which describes the probability, that a given event belongs to the class **signal**. This is done by minimizing the loss function (for example).
- **Various algorithms differ by:**
  - the class of function used as *f(x)* (linear, non-linear etc),
  - loss function
  - and the way it's minimized.
- **<u>Side remark</u>**: all these algorithms try to approximate the unknown **Bayesian Decisive Function** (BDF) relying on the finite training sample.
- **BDF** -an ideal classification function given by the unknown probability densities of signal and background.

Types of algorithms

- **Classification example:** find a function *f(x1,x2)* giving the probability, that a given data point belongs to a given class (signal vs background)



Remark on classification

- **Bayes classifier (optimal classifier**

$$P(S|x) = \frac{P(x|S)P(S)}{P(x|S)P(S) + P(x|B)P(B)}$$

  where **S** (signal) is associated with $y = 1$ and **B** (background) with $y = 0$. Bayes classifier accepts events x if P **(***S***|***x*) > cut as belonging to **S**.
- We need to approximate *the unknown* probability distributions *P(*x|S) and *P(x|B).*
- If your goal is to classify objects with the fewest errors, then the **Bayes classifier** is the optimal solution.
- **Consequently, if you have a classifier known to be *close* to the __Bayes limit__ , then *any* other classifier, *however sophisticated* , can *at best* be only marginally better than the one you have.**
  - Remark: If your problem is linear you don't gain anything by using the sophisticated Neural Network
- *All* classification methods, such as all we will be talking about, are different numerical approximations of the Bayes classifier.

Classification Accuracy

- Classification accuracy is the ratio of correct predictions to total predictions made.
  - **classification accuracy = correct predictions / total predictions**
- It is often presented as a percentage by multiplying the result by 100.
  - **classification accuracy = correct predictions / total predictions * 100**
- Classification accuracy can also easily be turned into a misclassification rate or error rate by inverting the value, such as:
  - **error rate = (1 - (correct predictions / total predictions)) * 100**

Confusion matrix

		Predicted Class: Positive	Predicted Class: Negative	
Actual Class	**Positive**	True Positive (TP)	False Negative (FN) **Type II Error**	**Sensitivity** $\frac{TP}{(TP+FN)}$
	Negative	False Positive (FP) **Type I Error**	True Negative (TN)	**Specificity** $\frac{TN}{(TN+FP)}$
		Precision $\frac{TP}{(TP+FP)}$	**Negative Predictive Value** $\frac{TN}{(TN+FN)}$	**Accuracy** $\frac{TP+TN}{(TP+TN+FP+FN)}$

Confusion matrix example

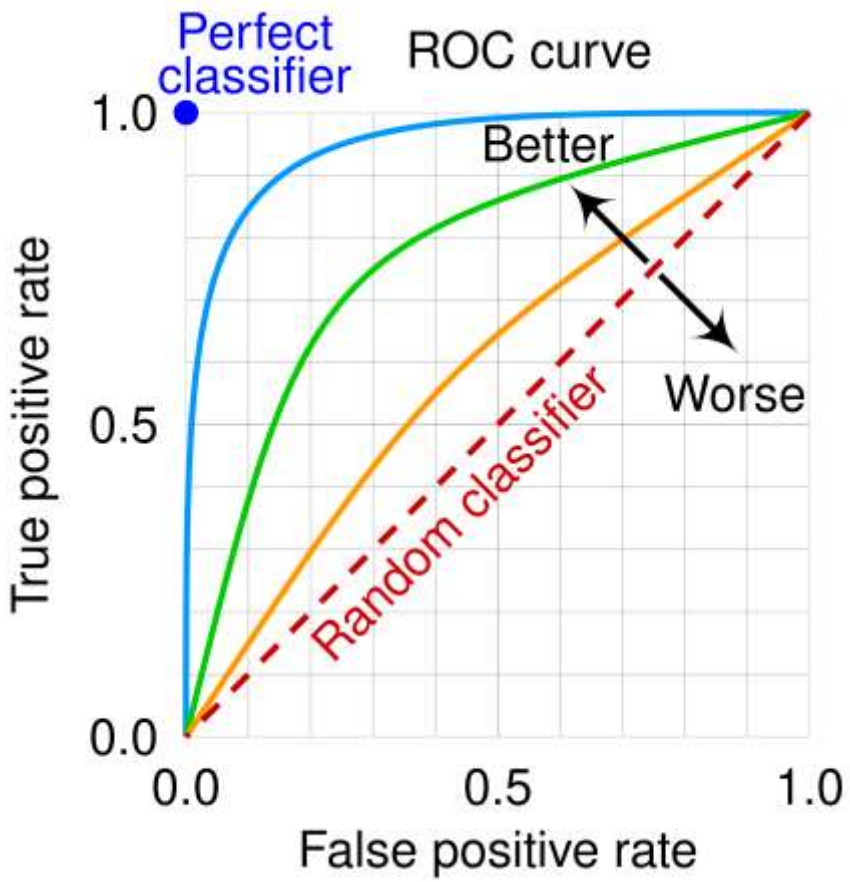
Predicted Values

Actual Values

	Setosa	Versicolor	Virginica	Row Total
Setosa	5	0	0	5
Versicolor	0	3	1	4
Virginica	0	1	5	6
Column Total	5	4	6	15

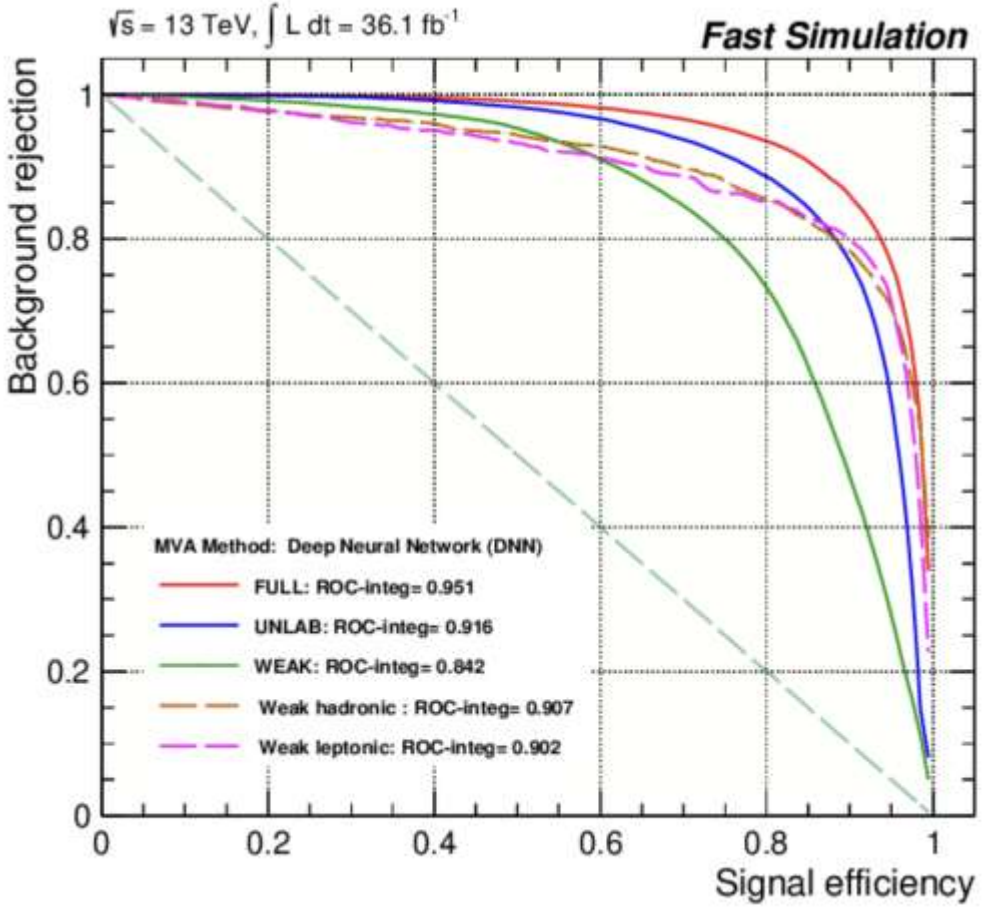
Goodness of classification

- **ROC** (Receiver Operation Characteristic) curve was first used to calibrate radars.
- Shows how good the classifier is.
- The **integral of ROC** could be a mea
  - Integral(ROC) = ½ - random classifier
  - Integral(ROC) = 1 - ideal classifier



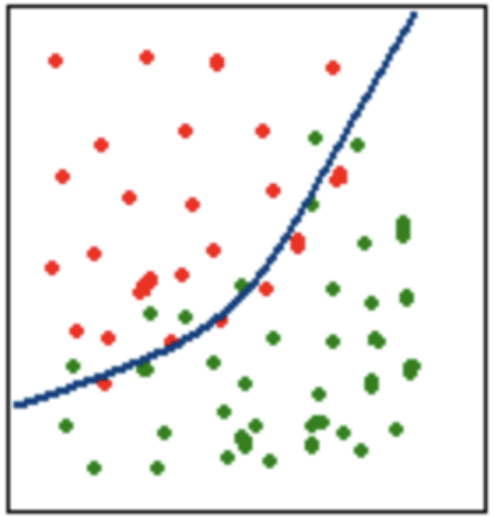
Goodness of classification

- **Alternative form of a ROC curve (used in particle physics)**
- Background rejection ($1-\varepsilon_B$) vs signal efficiency ε_S.

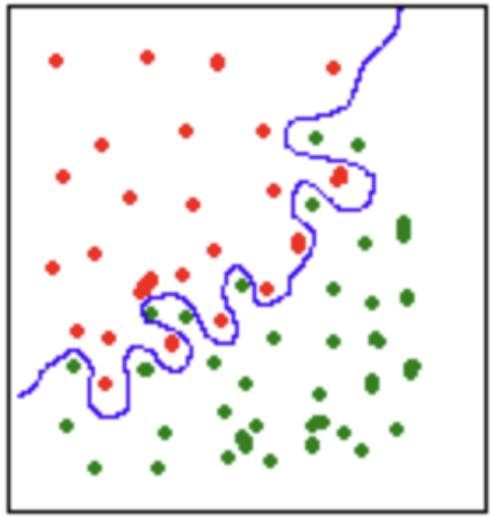


Overtraining

- **algorithm “learns” the particular events, not the rules.**
- **This effect is important for most of the ML algorithms.**



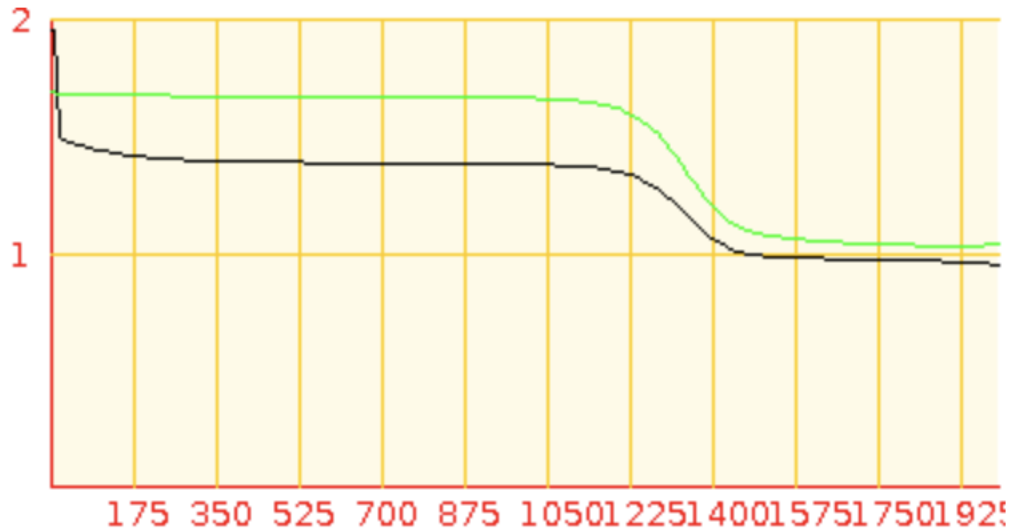
•classifier properly trained



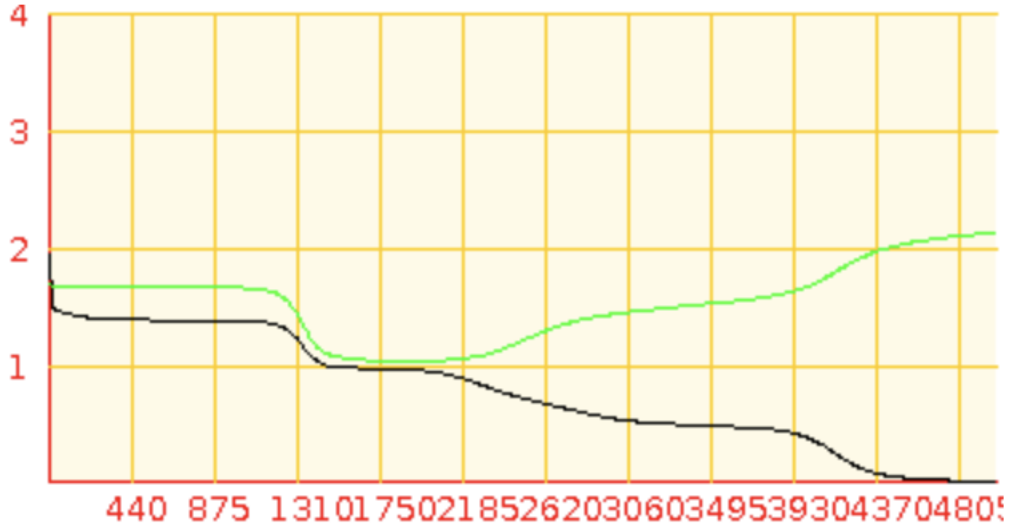
•**overtraining**. Instead of learning some general rules classifier "memorizes" events from the training dataset. For another statistically independent dataset the performance would be much worse.

- **Remedy – checking with another, independent dataset.**

Proper training vs overtraining



Proper training.
During training the loss is similar for both training and validation dataset.



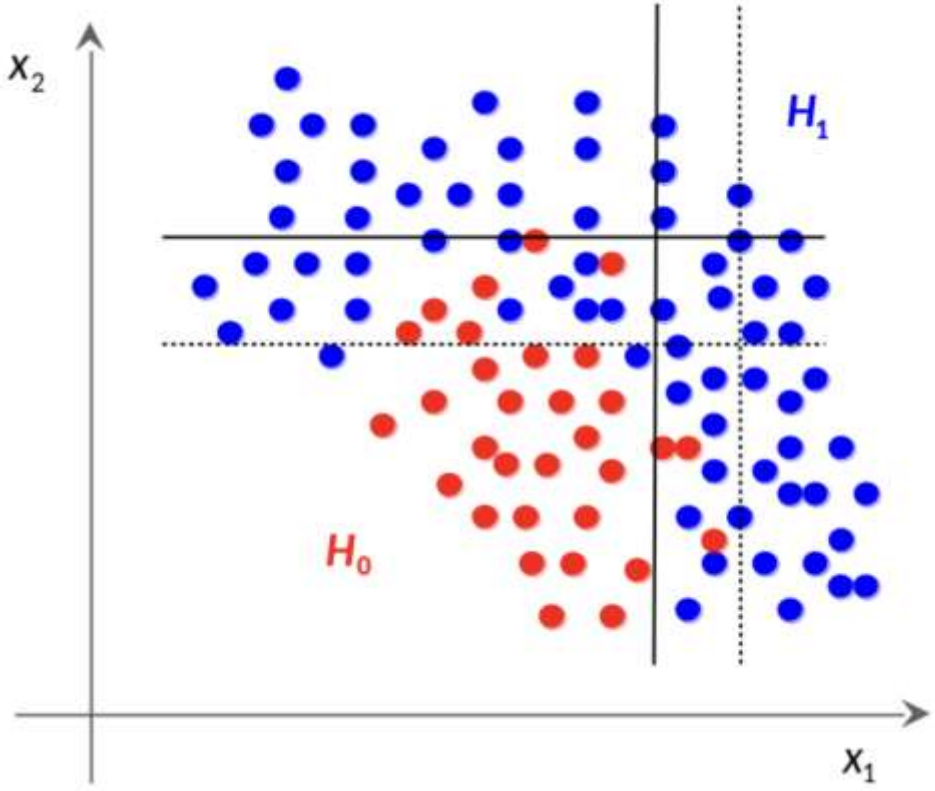
Overtraining. We should stop training before the overtraining appears (approximately where the losses start to be different)

Multivariate methods

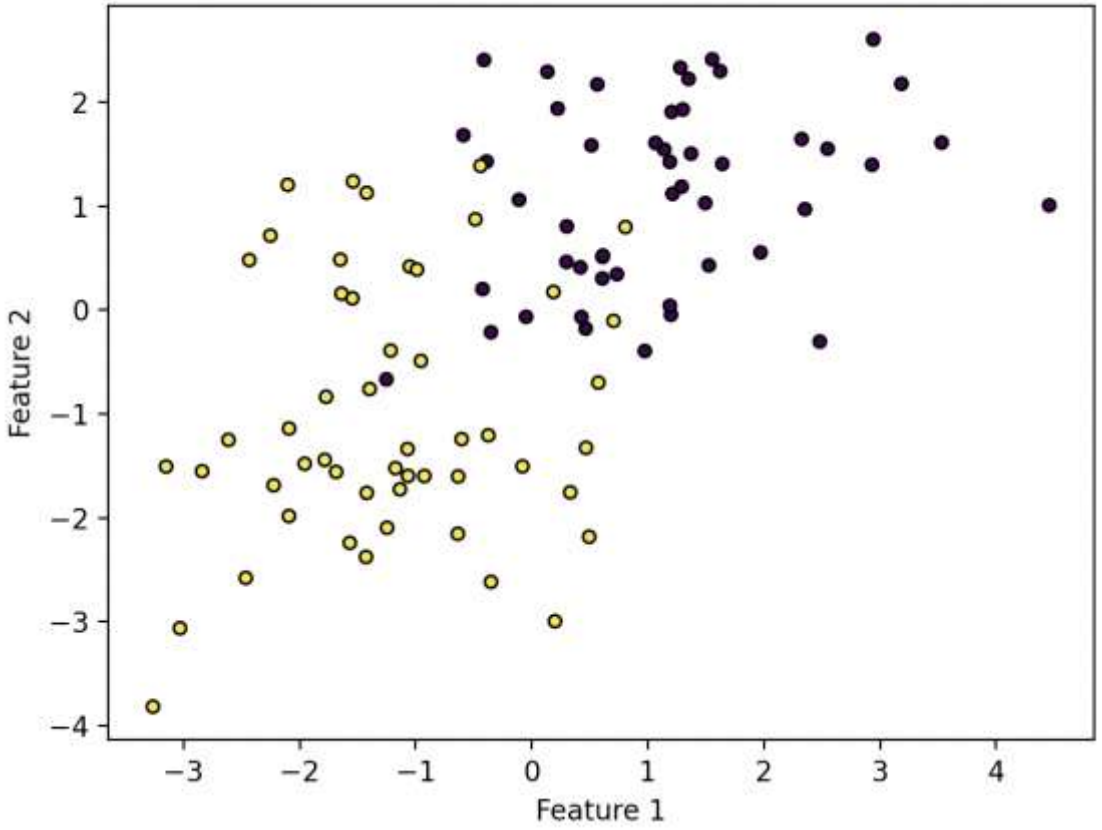
- **Short list of popular multivariate methods:**
  - Cuts
  - Linear Discriminants (like Fisher)
  - Naive Bayes (Likelihood Discriminant)
  - Kernel Density Estimation
  - Decision Trees
  - Neural Networks
  - Bayesian Neural Networks
  - Genetic Algorithms
  - Support Vector Machines
  - And many others...
- **We will talk first about simple ML linear methods**:
  - Cuts
  - Fisher linear discriminant
  - Naive Bayes
  - Principal Component Analysis (PCA)

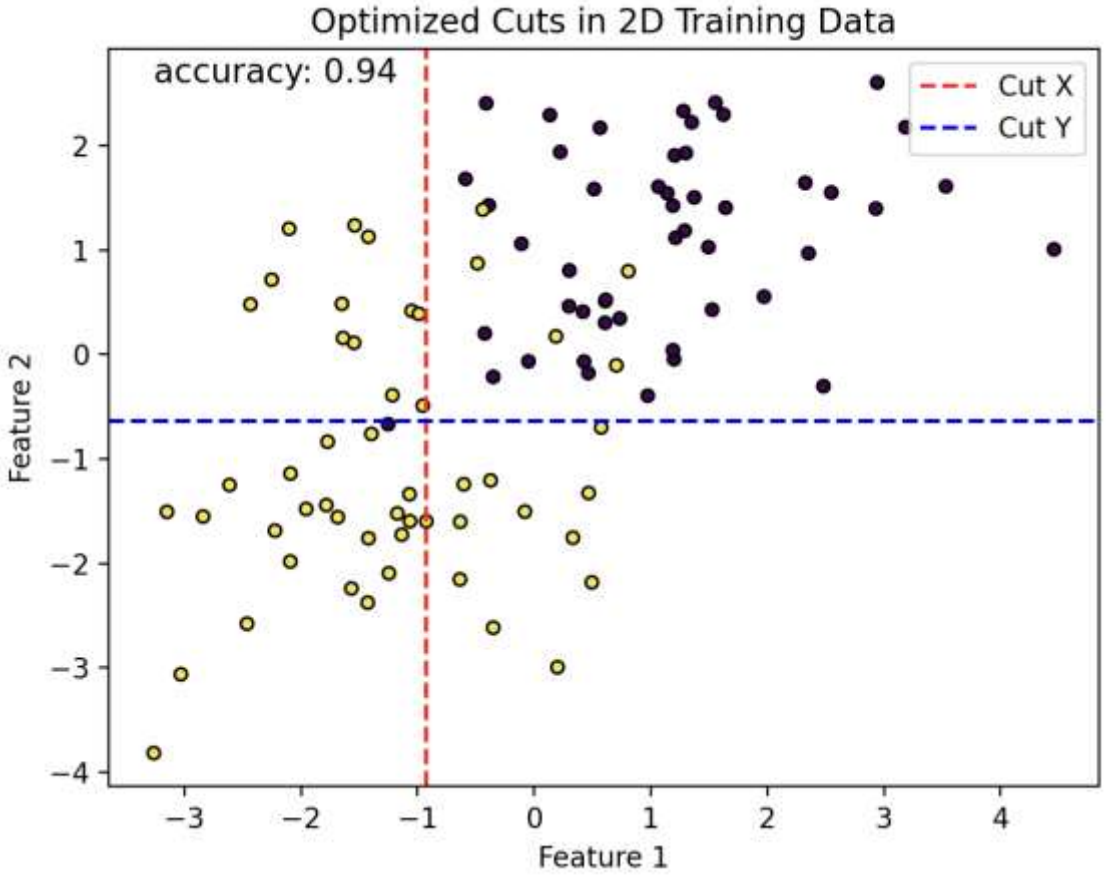
Selection cuts

- **Optimization of cuts:**
- Move cuts as long as we get the optimal signal vs. background selection. For a given signal efficiency we find the best background rejection → we get the entire ROC curve.
- Optimization methods:
  - Brute force
  - Genetic algorithms
  - Many others...



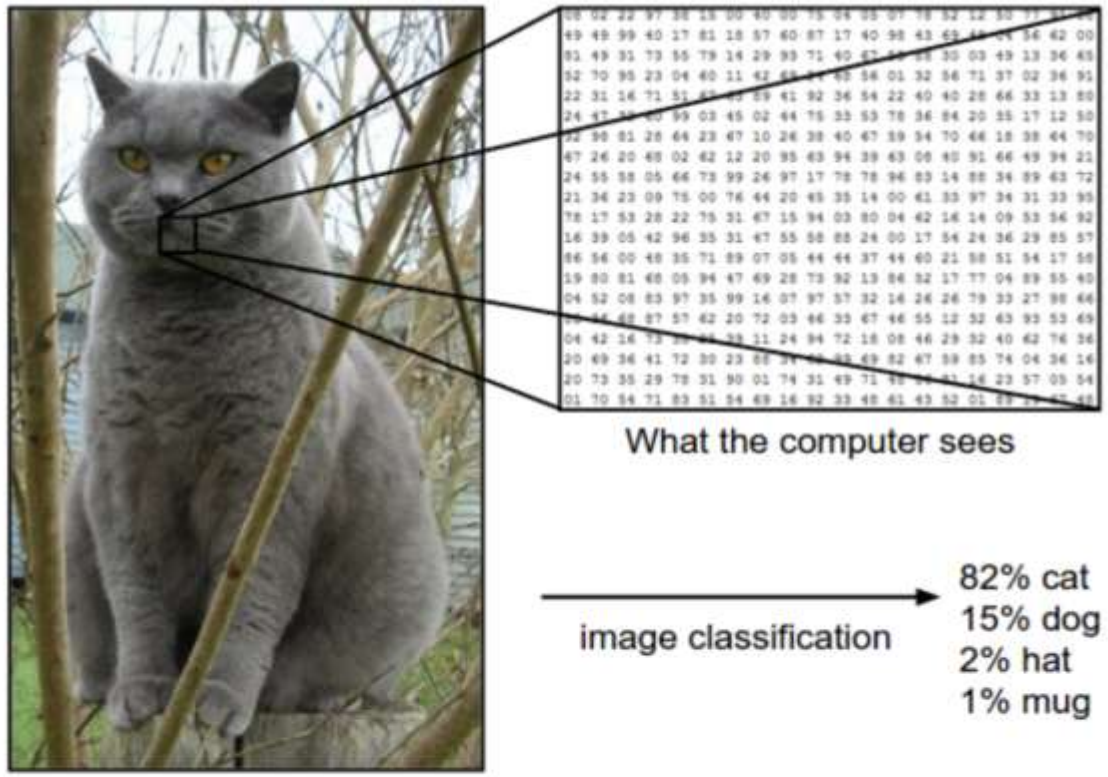
Selection cuts example





Classification in higher-dimensions

- The real “power” of AI methods is visible for large-dimensional problems
  - Greatest example: **image recognition**

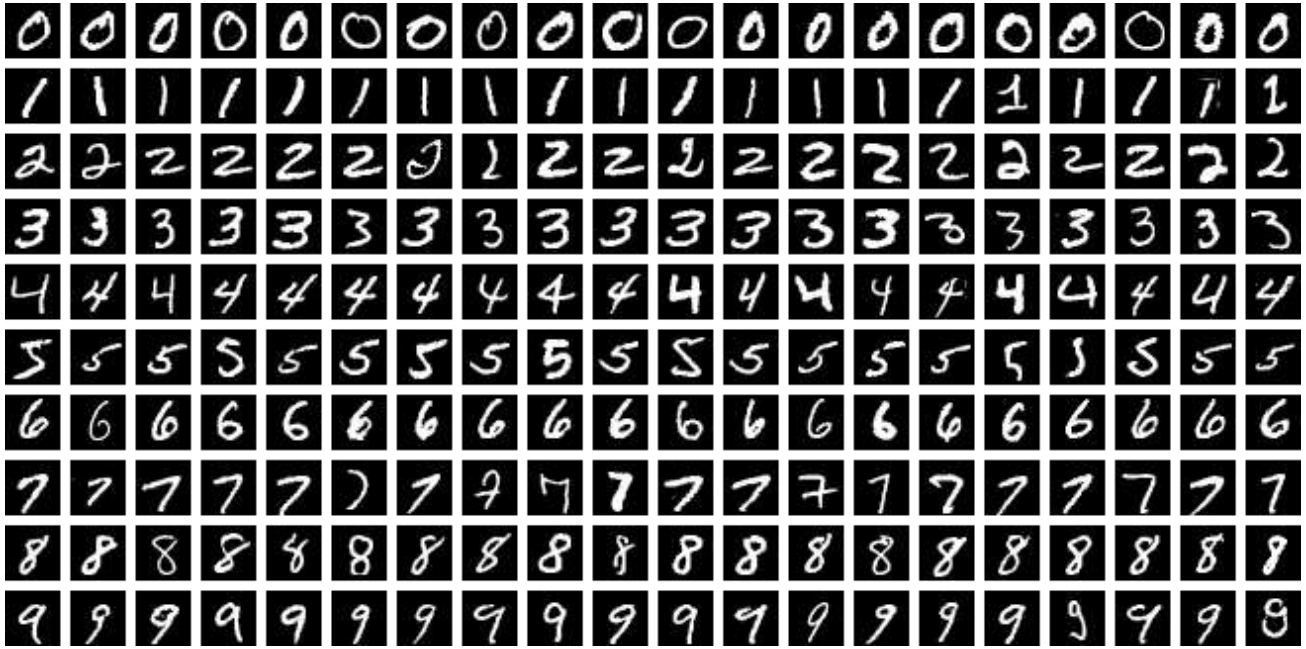


The task in Image Classification is to predict a single label (or a distribution over labels as shown here to indicate our confidence) for a given image. Images are 3-dimensional arrays of integers from 0 to 255, of size Width x Height x 3. The 3 represents the three color channels Red, Green, Blue.

Example image classification datasets

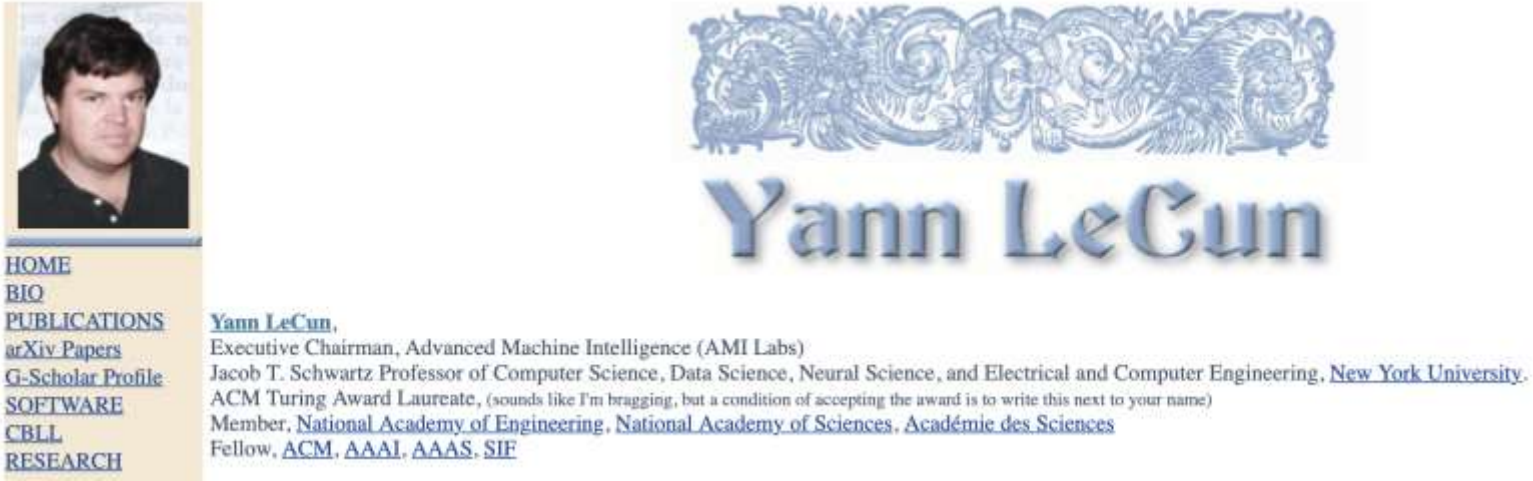
https://www.openml.org/d/554

- MNIST database
  - 70,000 images (60,000 training and 10,000 testing) of handwritten digits (zero to nine) that have been size-normalized and centered in a square grid of pixels
  - Each image is a 28 × 28 × 1 array of numbers (0-255) representing grayscale intensities ranging from 0 (black) to 1 (white).
  - The target data consists of one-hot binary vectors of size 10, corresponding to the digit classification categories zero through nine.



Example image classification datasets

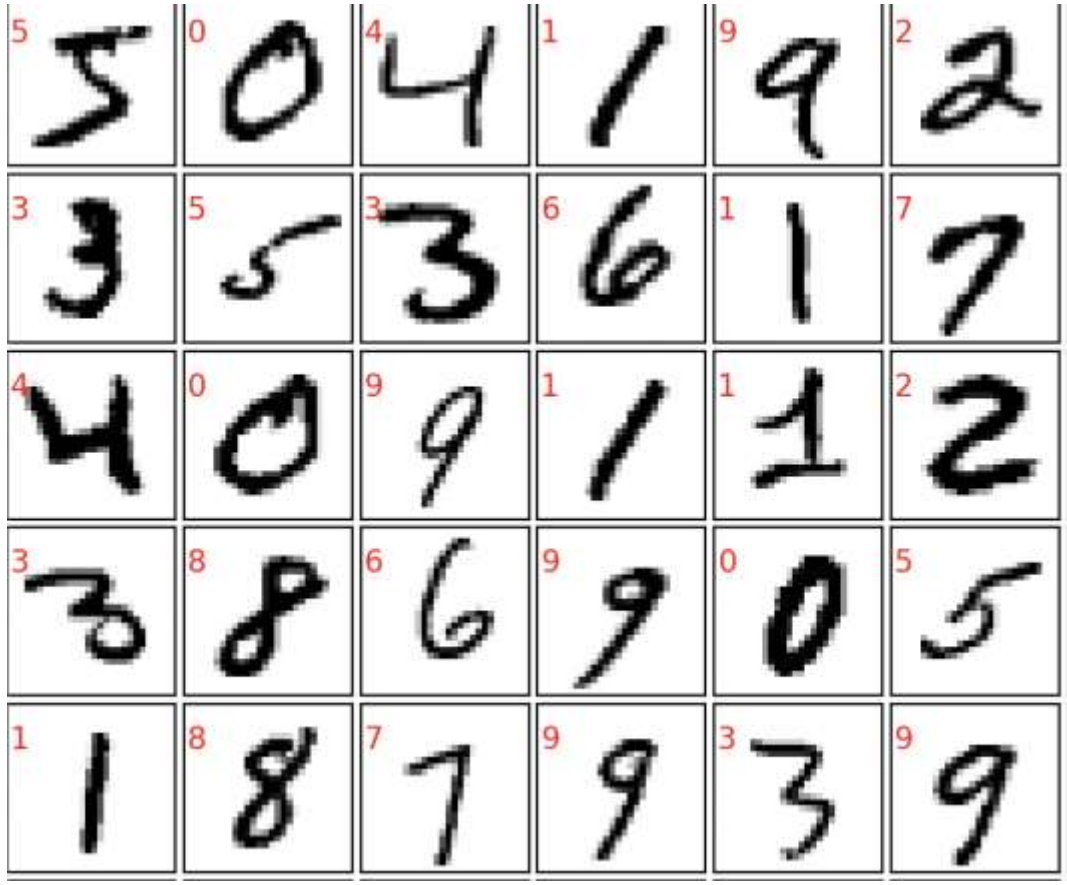
- **MNIST** database
  - Developed in the 1990s by Yann LeCun, Corinna Cortes, and Christopher J.C. Burges at AT&T Bell Labs
  - The dataset was derived from the National Institute of Standards and Technology's (NIST) Special Databases 1 (SD-1) and 3 (SD-3), which originally contained binary images of digits collected from high school students (SD-1) and Census Bureau employees (SD-3).
  - To address distributional differences between the original NIST training and test sets—where SD-3 proved easier due to its more uniform writing styles—the creators remixed samples from both databases



Example image classification datasets

- **MNIST** dataset with sklearn

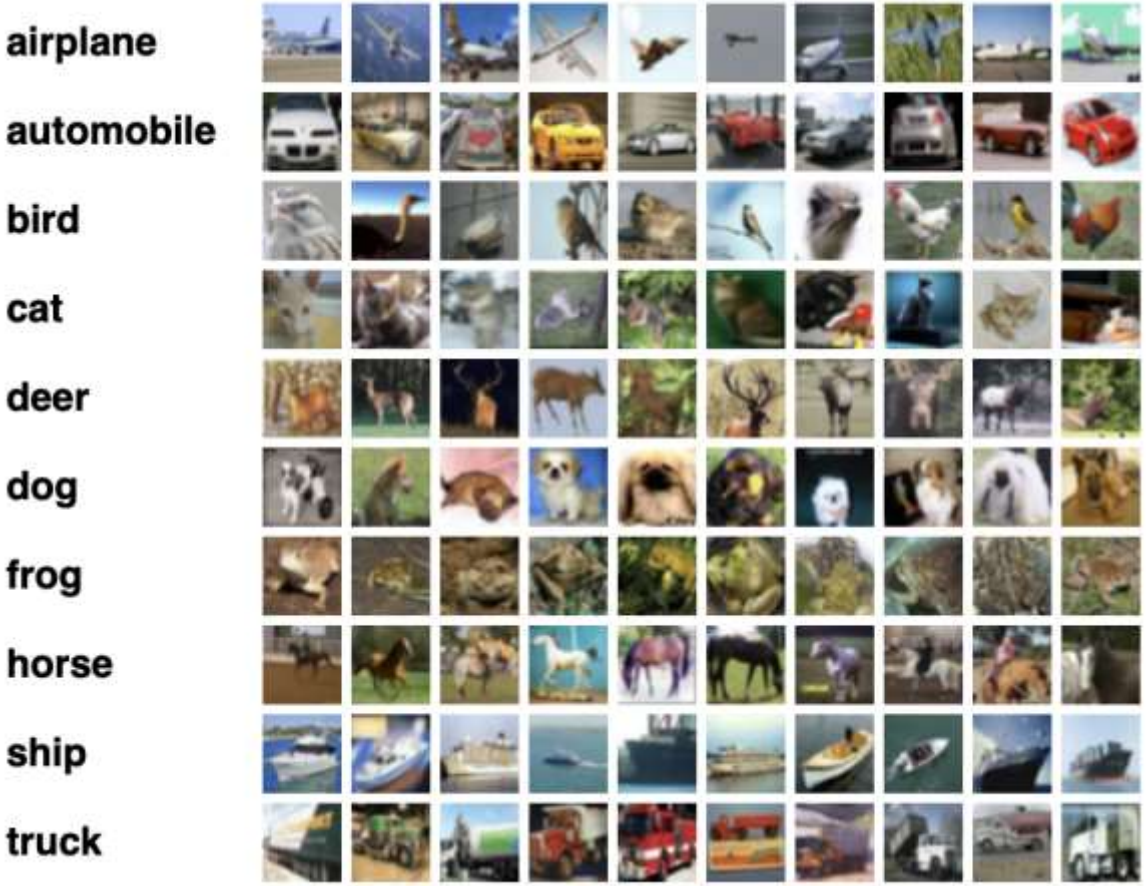
```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_openml

# Load MNIST data (28x28 images)
X, y = fetch_openml('mnist_784', version=1, return_X_y=True)

# Convert to NumPy and normalize
X = X.to_numpy() / 255.0
y = y.astype(int)

# Plot first 100 images
fig = plt.figure(figsize=(8, 8))
fig.subplots_adjust(
    left=0, right=1, bottom=0, top=1,
    hspace=0.05, wspace=0.05
)

for i in range(100):
    ax = fig.add_subplot(10, 10, i + 1, xticks=[], yticks=[])
    ax.imshow(X[i].reshape(28, 28),
              cmap=plt.cm.binary,
              interpolation='nearest')
    ax.text(0, 7, str(y[i]), color='red')

plt.show()
```



Example image classification datasets

- **CIFAR-10** (Canadian Institute For Advanced Research)
  - 60,000 tiny images that are 32 pixels high and wide.
  - Each image is labeled with one of 10 classes (for example, airplane, automobile, bird, etc).
  - These 60,000 images are partitioned into a training set of 50,000 images and a test set of 10,000 images.
  - Developed for evaluation of deep generative models in 2009
  - Students were employed to label the images, ensuring reliable and accurate annotations

https://www.cs.toronto.edu/~kriz/c

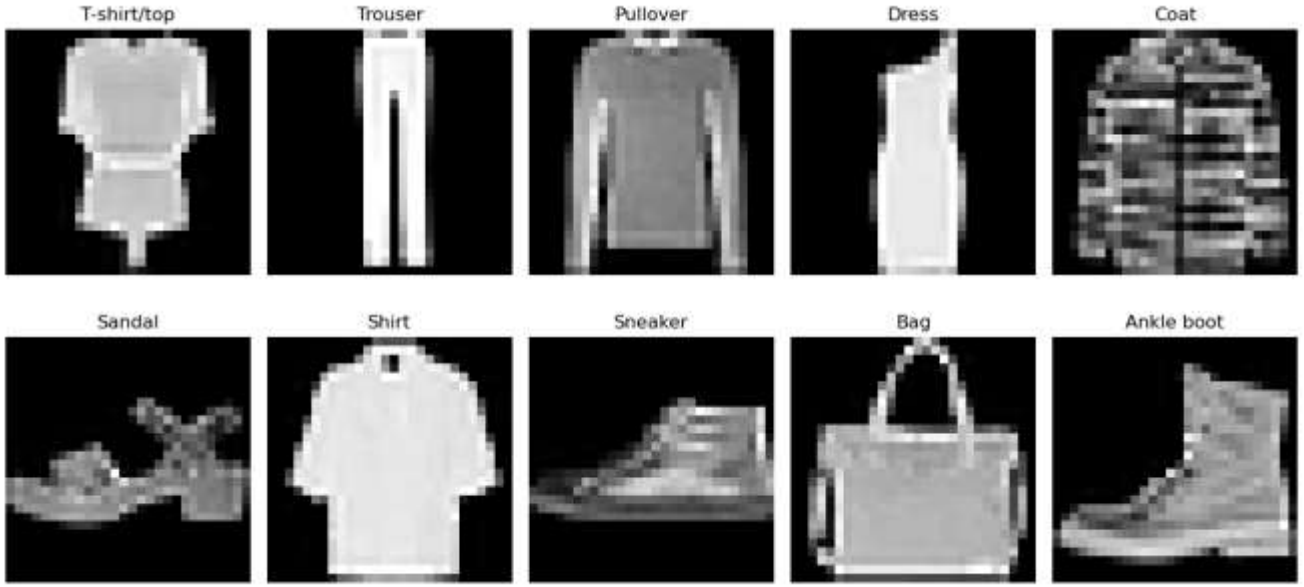


Example image classification datasets

- **CIFAR-10** dataset with keras

```python
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import cifar10

# Load data
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
y_train = y_train.flatten()

# Class names
class_names = [
    'airplane', 'automobile', 'bird', 'cat', 'deer',
    'dog', 'frog', 'horse', 'ship', 'truck'
]

num_classes = 10

# Plot one random image per class
fig = plt.figure(figsize=(10, 4))

for i in range(num_classes):
    ax = fig.add_subplot(2, 5, i + 1, xticks=[], yticks=[])

    # indices of images belonging to class i
    idx = np.where(y_train == i)[0]

    # randomly choose one image
    img_num = np.random.choice(idx)

    im = x_train[img_num]  # (32, 32, 3)
    ax.imshow(im)
    ax.set_title(class_names[i], fontsize=9)

plt.tight_layout()
plt.show()
```

Example image classification datasets

- **Fashion-MNIST** https://keras.io/api/datasets/fashion_mnist/
  - dataset of Zalando's article images
  - 60,000 28x28 grayscale images of 10 fashion categories, along with a test set of 10,000 images.
  - This dataset can be used as a drop-in replacement for MNIST.

```
keras.datasets.fashion_mnist.load_data()
```